

Transformer Architecture Interview Questions and Answers

Q1 [Beginner]: What is a Transformer model?

Answer: A Transformer is a neural network architecture that uses self-attention to process sequences in parallel, making it highly efficient for NLP tasks.

Q2 [Beginner]: Why was the Transformer architecture introduced?

Answer: Transformers were introduced to replace sequential RNN processing with parallel self-attention, improving speed and handling long-range dependencies.

Q3 [Beginner]: What is self-attention?

Answer: Self-attention enables each token to focus on relevant tokens in the sequence to capture contextual relationships.

Q4 [Beginner]: What are Query, Key, and Value in self-attention?

Answer: Q, K, and V are learned vectors used in self-attention where Queries match Keys to determine attention weights applied to Values.

Q5 [Beginner]: What is positional encoding?

Answer: Positional encoding adds token position information so Transformers can understand word order.

Q6 [Beginner]: What is Multi-Head Attention?

Answer: Multi-Head Attention allows the model to learn multiple contextual relationships simultaneously using several attention heads.

Q7 [Beginner]: What is the Encoder in a Transformer?

Answer: The encoder converts input tokens into contextual representations using self-attention and feed-forward layers.

Q8 [Beginner]: What is the Decoder in a Transformer?

Answer: The decoder generates outputs by attending to previous tokens and encoder representations.

Q9 [Beginner]: What is Masked Self-Attention?

Answer: Masked Self-Attention blocks access to future tokens, ensuring causal text generation.

Q10 [Beginner]: What is the overall flow of a Transformer?

Answer: Input tokens → Embeddings + Positional Encoding → Encoder → Decoder → Linear Layer → Softmax → Output Token.

Q11 [Intermediate]: How is the Attention Score calculated in a Transformer?

Answer: Attention scores are computed using QK^T , scaled by $\sqrt{d_k}$, passed through Softmax, and applied to the Value vectors.

Q12 [Intermediate]: Why is the scaling factor $\sqrt{d_k}$ used in Scaled Dot-Product Attention?

Answer: The $\sqrt{d_k}$ scaling factor prevents large attention scores and stabilizes Softmax gradients during training.

Q13 [Intermediate]: What is the difference between Self-Attention and Cross-Attention?

Answer: Self-Attention attends within the same sequence, while Cross-Attention attends to a different sequence, typically encoder outputs.

Q14 [Intermediate]: What are Residual Connections and why are they used?

Answer: Residual Connections add layer inputs to outputs, improving gradient flow and enabling deeper networks.

Q15 [Intermediate]: What is Layer Normalization in Transformers?

Answer: Layer Normalization stabilizes training by normalizing features for each token independently.

Q16 [Intermediate]: What is the Feed Forward Network (FFN) in a Transformer?

Answer: The FFN is a two-layer neural network applied after attention to learn richer token representations.

Q17 [Intermediate]: How does BERT differ from GPT?

Answer: BERT is encoder-only and bidirectional for understanding tasks, while GPT is decoder-only and autoregressive for text generation.

Q18 [Intermediate]: What is the difference between Encoder-Only, Decoder-Only, and Encoder-Decoder architectures?

Answer: Encoder-Only models understand text, Decoder-Only models generate text, and Encoder-Decoder models perform sequence-to-sequence tasks.

Q19 [Intermediate]: What is Causal Attention?

Answer: Causal Attention prevents tokens from attending to future positions, enabling autoregressive text generation.

Q20 [Intermediate]: What is the computational complexity of Self-Attention?

Answer: Self-Attention has $O(n^2)$ complexity because every token interacts with every other token.

Q21 [Advanced]: What is Flash Attention?

Answer: Flash Attention is a memory-efficient attention algorithm that computes attention in blocks, reducing memory usage and speeding up Transformer training and inference.

Q22 [Advanced]: What is KV Cache and why is it important in LLM inference?

Answer: KV Cache stores previously computed Key and Value tensors, avoiding recomputation and greatly accelerating LLM inference.

Q23 [Advanced]: What are Rotary Positional Embeddings (RoPE)?

Answer: RoPE encodes position through rotational transformations of Query and Key vectors, improving long-context performance.

Q24 [Advanced]: What is ALiBi (Attention with Linear Biases)?

Answer: ALiBi adds linear distance-based biases to attention scores, helping models generalize to longer contexts.

Q25 [Advanced]: What is Sparse Attention?

Answer: Sparse Attention limits token interactions to selected tokens, reducing computation and enabling longer context processing.

Q26 [Advanced]: What is Multi-Query Attention (MQA)?

Answer: MQA uses multiple Query heads with shared Key and Value heads to reduce memory usage and improve inference speed.

Q27 [Advanced]: What is Grouped Query Attention (GQA)?

Answer: GQA groups Query heads to share Key and Value heads, balancing model quality and inference efficiency.

Q28 [Advanced]: What are Transformer Scaling Laws?

Answer: Scaling Laws explain how increasing parameters, data, and compute leads to predictable improvements in Transformer performance.

Q29 [Advanced]: What is a Mixture of Experts (MoE) architecture?

Answer: MoE activates only selected expert networks for each token, enabling massive model scaling with efficient computation.

Q30 [Advanced]: How do modern LLMs support long-context windows?

Answer: Modern LLMs use techniques like RoPE, ALiBi, Flash Attention, Sparse Attention, and KV Cache optimizations to support long-context windows.

Q31 [Expert]: How does the KV Cache reduce inference latency in large language models?

Answer: KV Cache stores previously computed Key and Value tensors, eliminating redundant attention computations and significantly reducing inference latency.

Q32 [Expert]: Why do modern LLMs prefer RoPE over traditional positional encodings?

Answer: RoPE embeds position information into attention calculations, enabling better long-context generalization than traditional positional encodings.

Q33 [Expert]: Compare MHA, MQA, and GQA in terms of memory and performance.

Answer: MHA provides highest quality but uses most memory, MQA minimizes memory usage, and GQA balances efficiency and performance.

Q34 [Expert]: How does Flash Attention achieve memory efficiency?

Answer: Flash Attention computes attention in memory-efficient blocks, avoiding storage of the full attention matrix and reducing memory bandwidth costs.

Q35 [Expert]: What are the challenges of extending context windows to 1M+ tokens?

Answer: The main challenges are quadratic attention cost, massive KV cache requirements, and maintaining attention quality over extremely long contexts.

Q36 [Expert]: How does Mixture of Experts (MoE) improve model scalability?

Answer: MoE scales model capacity by activating only selected experts per token, enabling massive parameter counts with efficient computation.

Q37 [Expert]: What are Transformer Scaling Laws and how do they influence model training?

Answer: Scaling Laws provide predictable relationships between parameters, data, compute, and performance, guiding efficient model training strategies.

Q38 [Expert]: How do models like GPT, Llama, and DeepSeek optimize Transformer architectures?

Answer: GPT, Llama, and DeepSeek optimize Transformers using techniques such as RoPE, GQA, MoE, efficient normalization, and advanced training strategies.

Q39 [Expert]: What causes attention bottlenecks in large-scale Transformers and how can they be mitigated?

Answer: Attention bottlenecks arise from $O(n^2)$ complexity and can be mitigated using Flash Attention, Sparse Attention, GQA, MQA, and retrieval-based methods.

Q40 [Expert]: How would you design a production-grade Transformer for billion-user scale applications?

Answer: Use Flash Attention, KV Cache, GQA/MQA, RoPE, MoE, distributed serving, autoscaling, monitoring, and safety controls to build a billion-user-scale Transformer system.

Q41 [Scenario]: Your Transformer model is taking too long to process 100K-token documents. How would you optimize it?

Answer: I would use Flash Attention, Sparse Attention, chunking, RAG, KV Cache optimization, and quantization to efficiently process 100K-token documents.

Q42 [Scenario]: A GPT-style model is consuming excessive GPU memory during inference. What techniques would you use to reduce memory usage?

Answer: I would reduce memory using quantization, GQA/MQA, KV Cache optimization, Flash Attention, model distillation, and efficient inference frameworks.

Q43 [Scenario]: You need to build a chatbot that remembers conversations across 500 pages of documentation. How would you design the architecture?

Answer: I would use a RAG architecture with vector search, document chunking, embeddings, and conversation memory management.

Q44 [Scenario]: Your LLM responses become slower as the conversation grows. What is causing this and how would you solve it?

Answer: Growing context windows increase attention computation. I would use summarization, memory management, retrieval, and KV Cache optimizations.

Q45 [Scenario]: You are asked to support a 1-million-token context window. What architectural changes would you consider?

Answer: I would use Sparse Attention, Flash Attention, retrieval systems, hierarchical memory, and context compression to support million-token contexts.

Q46 [Scenario]: A production Transformer model is experiencing latency spikes during peak traffic. How would you investigate and fix the issue?

Answer: I would analyze infrastructure metrics, identify bottlenecks, and mitigate them using autoscaling, batching, caching, and load balancing.

Q47 [Scenario]: You need to serve an LLM to millions of users while keeping infrastructure costs low. What optimizations would you implement?

Answer: I would optimize costs using quantization, batching, caching, model routing, GQA/MQA, autoscaling, and efficient serving frameworks.

Q48 [Scenario]: A customer wants real-time translation with very low latency. Which Transformer architecture would you choose and why?

Answer: I would use an Encoder-Decoder Transformer optimized with quantization, batching, and GPU acceleration for low-latency translation.

Q49 [Scenario]: Your model performs well during training but poorly in production. How would you diagnose the issue?

Answer: I would check for data drift, distribution mismatch, retrieval issues, prompt differences, and production monitoring metrics.

Q50 [Scenario]: Design a GenAI system that answers enterprise questions using internal documents while minimizing hallucinations.

Answer: I would build a RAG system with vector search, reranking, grounded prompts, citations, guardrails, and evaluation metrics to reduce hallucinations.